

```

5      # 1. 保存到哪里？内核栈？专用区域？
6      # 2. 保存哪些寄存器？
7      # 3. 如何快速保存和恢复？
8
9      # 调用C处理函数
10     call kerneltrap
11
12     # 恢复上下文并返回

```

任务5：实现时钟中断与调度

时钟中断处理：

1. 理解SBI时钟接口：

```

1      // 设置下次时钟中断时间
2      void sbi_set_timer(uint64 time);
3      // 获取当前时间
4      uint64 get_time(void);

```

2. 实现时钟中断处理函数：

```

1      void timer_interrupt(void) {
2          // 1. 更新系统时间
3          // 2. 处理定时器事件
4          // 3. 触发任务调度
5          // 4. 设置下次中断时间
6      }

```

调度器集成：

- 如何在时钟中断中触发调度？
- 调度的时机选择有什么考虑？
- 如何确保调度的原子性？

任务6：异常处理机制

异常类型理解：

1. 指令地址未对齐
2. 指令访问故障
3. 非法指令
4. 断点
5. 加载地址未对齐
6. 加载访问故障
7. 存储地址未对齐
8. 存储访问故障
9. 用户模式环境调用
10. 监督模式环境调用

实现要求:

```
1 void handle_exception(struct trapframe *tf) {
2     uint64 cause = r_scause();
3
4     switch (cause) {
5     case 8: // 系统调用
6         handle_syscall(tf);
7         break;
8     case 12: // 指令页故障
9         handle_instruction_page_fault(tf);
10        break;
11    case 13: // 加载页故障
12        handle_load_page_fault(tf);
13        break;
14    case 15: // 存储页故障
15        handle_store_page_fault(tf);
16        break;
17    default:
18        panic("Unknown exception");
19    }
20 }
```

测试与调试策略

中断功能测试

```
1 void test_timer_interrupt(void) {
2     printf("Testing timer interrupt...\n");
3
4     // 记录中断前的时间
5     uint64 start_time = get_time();
6     int interrupt_count = 0;
7
8     // 设置测试标志
9     volatile int *test_flag = &interrupt_count;
10
11    // 在时钟中断处理函数中增加计数
12    // 等待几次中断
13    while (interrupt_count < 5) {
14        // 可以在这里执行其他任务
15        printf("Waiting for interrupt %d...\n", interrupt_count + 1);
16        // 简单延时
17        for (volatile int i = 0; i < 1000000; i++);
18    }
19
20    uint64 end_time = get_time();
21    printf("Timer test completed: %d interrupts in %lu cycles\n",
```

```
22         interrupt_count, end_time - start_time);
23     }
```

异常处理测试

```
1 void test_exception_handling(void) {
2     printf("Testing exception handling...\n");
3
4     // 测试除零异常 (如果支持)
5     // 测试非法指令异常
6     // 测试内存访问异常
7
8     printf("Exception tests completed\n");
9 }
```

性能测试

```
1 void test_interrupt_overhead(void) {
2     // 测量中断处理的时间开销
3     // 测量上下文切换的成本
4     // 分析中断频率对系统性能的影响
5 }
```

调试建议

分阶段调试

1. **基础设置验证:**
 - 验证中断寄存器设置是否正确
 - 检查中断向量地址是否对齐
 - 确认中断使能位设置
2. **中断触发测试:**
 - 使用简单的时钟中断测试
 - 在中断处理函数中添加输出确认被调用
 - 验证中断频率是否符合预期
3. **上下文完整性:**
 - 在中断前后检查寄存器值
 - 验证栈指针的正确性
 - 确认中断返回后程序继续正常执行

常见问题诊断

问题：中断无响应

- 检查中断使能位设置

- 验证中断向量地址
- 确认中断源是否正确配置

问题：系统在中断处理后崩溃

- 检查栈指针保存和恢复
- 验证上下文保存的完整性
- 确认中断处理函数没有破坏调用约定

问题：中断频率异常

- 检查时钟设置参数
- 验证SBI调用是否正确
- 确认时间计算没有溢出

思考题

1. 中断设计：

- 为什么时钟中断需要在M模式处理后再委托给S模式？
- 如何设计一个支持中断优先级的系统？

2. 性能考虑：

- 中断处理的时间开销主要在哪里？如何优化？
- 高频率中断对系统性能有什么影响？

3. 可靠性：

- 如何确保中断处理函数的安全性？
- 中断处理中的错误应该如何处理？

4. 扩展性：

- 如何支持更多类型的中断源？
- 如何实现中断的动态路由？

5. 实时性：

- 当前实现的中断延迟特征如何？
 - 如何设计一个满足实时要求的中断系统？
-

实验5：进程管理与调度

实验目标

通过深入分析xv6的进程管理机制，理解操作系统如何创建、管理和调度进程，实现完整的进程生命周期管理和简单的调度算法。

核心学习资料